Verilog RTL Design and RISC-V Processor Development

Day 2: FSM Controllers and Memory Design | Day 3: Extending the Single-Cycle RISC-V CPU

Core activities 2.1 to 2.4 and 3.1 to 3.4. Each section gives the design approach, verification method, simulation results, waveform, issues faced, and full source listings.

Tools. Icarus Verilog 12.0 (`iverilog -g2012`, `vvp`), Yosys (synthesis and latch check), Python 3 (assembler, instruction-set simulator, waveform rendering). Waveforms are exported from the simulator's VCD files; open them in GTKWave for interactive viewing.
Assumptions (please read). The workbook PDF was not available when this was prepared, so the exact specifications were inferred from the activity titles. Choices made: sequence **1011** (overlapping); vending price **15 cents** with nickel/dime inputs; FIFO **8 x 16**; CPU is a **from-scratch RV32I single-cycle design** (the workbook's own base CPU was not seen), so '3.1 instruction extensions' lists what was added on top of a minimal base subset. If the workbook specifies different parameters or instructions, these are easy to change.

Activity	Result
2.1 Sequence Detector	3033 cycles, 183 detections, 0 errors -> PASS
2.2 Vending Machine	4039 cycles, 726 dispenses, 267 change, 0 errors -> PASS
2.3 Synchronous FIFO	(DATA_W=8 DEPTH=16): 5084 cycles, 1646 writes, 1644 reads, 927 overflow pulses, 992 underflow pulses, 0 errors -> PASS (DATA_W=8 DEPTH=4): 5030 cycles, 1517 writes, 1515 reads, 1014 overflow pulses, 1085 underflow pulses, 0 errors -> PASS (DATA_W=16 DEPTH=32): 5156 cycles, 1727 writes, 1725 reads, 902 overflow pulses, 959 underflow pulses, 0 errors -> PASS
2.4 Design Review	Lint, synthesis (no latches), mutation analysis (8/8 injected bugs caught); findings in section 2.4
3.1 Instruction Extensions	15 instructions added (plus 6 branch/jump instructions from 3.5) on top of a 10-instruction base; 31 RV32I instructions in total
3.2 Assembly Programs	5 programs, all verified against known answers
3.3 Self-Checking CPU Testbench	5 programs, 0 failed, 0 total mismatches -> PASS; 12/12 injected RTL bugs caught
3.4 Waveform Analysis	Cycle-by-cycle trace of prog_sum, CPI = 1 confirmed

2.1 Sequence Detector

Design approach. A Mealy FSM detects the bit pattern **1011** on a serial input, with overlap allowed (the final 1 of one match starts the next). Four states record the longest suffix of the input that is a prefix of the pattern: S0 none, S1 '1', S2 '10', S3 '101'. From S3, a 1 completes the pattern and moves to S1 (trailing 1 is a new prefix); a 0 means the input was 1010, so the machine moves to S2 ('10'). The output is `(state==S3) & din`, which asserts in the same cycle the last bit arrives. Synchronous active-high reset.

Verification. A reference model keeps a 4-bit shift register of recent inputs and flags a match when the last four bits equal 1011; the DUT output is compared every cycle (sampled at the posedge, before the update, to avoid races). Directed tests: overlapping stream 01011011 must give exactly 2 hits; a near-miss stream must give 0; a reset in the middle of a pattern (101, reset, 1) must give 0. Then 3000 cycles of random input with random resets.

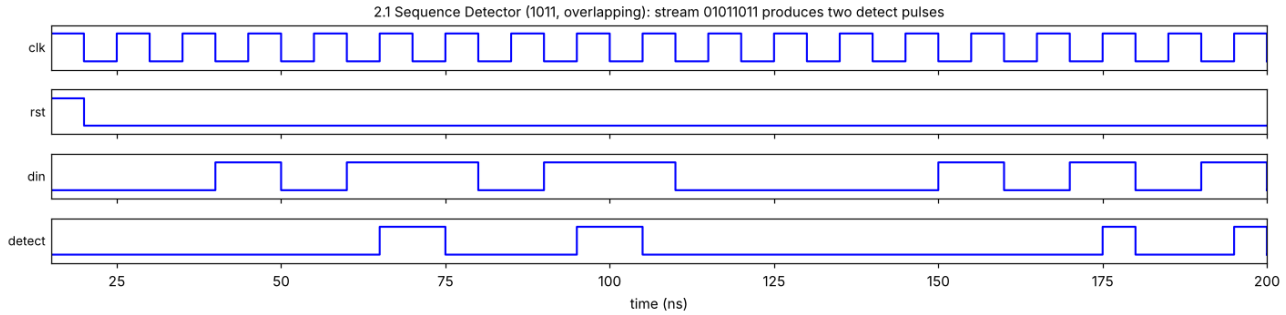
Result. 3033 cycles, 183 detections, 0 errors. Mutation check: changing the S3 fall-back (1010 to S0) caused 39 errors; dropping `& din` from the output caused 177 errors, so the testbench is sensitive to both overlap handling and output timing.

Issues / notes. The Mealy output depends combinationally on `din`; if the input arrives late or glitches, so can the output. A registered output would fix that at the cost of one cycle of latency (see 2.4).

Simulation output

```
directed overlap stream 01011011 -> hits=2 (expect 2)
directed near-miss stream -> hits=0 (expect 0)
reset mid-pattern -> hits=0 (expect 0)
SEQ_DETECTOR TEST: 3033 cycles, 183 detections, 0 errors -> PASS
```

Waveform



2.1 RTL source (seq_detector.v)

```
// 2.1 Sequence Detector: overlapping "1011", Mealy FSM, synchronous active-high reset
module seq_detector (
    input  wire clk, rst, din,
    output wire detect
);
    localparam S0 = 2'd0,   // no useful prefix
               S1 = 2'd1,   // seen "1"
               S2 = 2'd2,   // seen "10"
               S3 = 2'd3;   // seen "101"
    reg [1:0] state, next;
    always @(posedge clk) state <= rst ? S0 : next;
    always @* begin
        case (state)
            S0: next = din ? S1 : S0;
            S1: next = din ? S1 : S2;
            S2: next = din ? S3 : S0;
            S3: next = din ? S1 : S2;   // "1011" done: trailing 1 is a new prefix; "1010" keeps "10"
            default: next = S0;
        endcase
    end
    assign detect = (state == S3) & din;  // Mealy output
endmodule
```

2.1 Testbench (seq_detector_tb.v)

```
`timescale 1ns/1ps
module seq_detector_tb;
    reg clk=0, rst=1, din=0; wire detect;
    seq_detector dut(clk,rst,din,detect);
    always #5 clk = ~clk;
    // reference model: 4-bit shift register of the last inputs
    reg [3:0] sh = 0; reg exp_detect; reg check_en = 0;
    integer errors=0, cycles=0, hits=0, i;
    always @(posedge clk) begin           // sample pre-edge values
        if (rst) begin sh = 0; check_en = 1; end
        else if (check_en) begin
            cycles = cycles + 1;
            exp_detect = ({sh[2:0], din} == 4'b1011);
            if (detect !== exp_detect) begin
                errors = errors + 1; $display("FAIL t=%0t din=%b detect=%b exp=%b", $time, din, detect, exp_detect);
            end
            if (detect) hits = hits + 1;
            sh = {sh[2:0], din};
        end
    end
    task send(input [15:0] bits, input integer n); integer k; begin
        for (k=n-1;k>=0;k=k-1) begin @(negedge clk); din = bits[k]; end
    end endtask
    integer hits_before;
    initial begin
        $dumpfile("seq.vcd"); $dumpvars(0, seq_detector_tb);
        repeat (2) @(negedge clk); rst = 0;
        // directed 1: overlapping occurrences  1011011 -> exactly 2 hits
        hits_before = hits; send(16'b0000_0000_0101_1011, 8); send(16'b0, 2);
        @(negedge clk); @(negedge clk);
        $display("directed overlap stream 01011011 -> hits=%0d (expect 2)", hits-hits_before);
        if (hits-hits_before != 2) errors = errors + 1;
        // directed 2: near misses must not fire
        hits_before = hits; send(16'b1010_1010_0110_0111, 16); send(16'b0, 2);
        @(negedge clk);
        $display("directed near-miss stream -> hits=%0d (expect 0)", hits-hits_before);
        if (hits-hits_before != 0) errors = errors + 1;
        // reset in the middle of a pattern: 101 | rst | 1 must NOT fire
        hits_before = hits; send(5'b101, 3); @(negedge clk); rst = 1; din = 0; @(negedge clk); rst = 0; send(1'b1, 1); send(16'b
        @(negedge clk);
        $display("reset mid-pattern -> hits=%0d (expect 0)", hits-hits_before);
        if (hits-hits_before != 0) errors = errors + 1;
        // random stream
        for (i=0;i<3000;i=i+1) begin @(negedge clk); din = $random; if ($urandom%500==0) rst=1; else rst=0; end
        @(negedge clk); @(negedge clk);
        $display("SEQ_DETECTOR TEST: %0d cycles, %0d detections, %0d errors -> %s", cycles, hits, errors, errors==0?"PASS":"FAIL
        $finish;
    end
endmodule
```

2.2 Vending Machine Controller

Design approach. Moore FSM for a 15-cent item accepting nickels (5c) and dimes (10c), one coin per cycle. States track the credit: C0 (0c), C5, C10, then two output states VEND (credit 15c, `dispense`) and VEND_CHG (credit 20c, `dispense` + `change` = return a nickel). Both output states last one cycle and return to C0. Coin inputs asserted together are invalid and ignored; coins arriving during the dispensing cycle are ignored (documented design decision). Moore outputs are decoded from the state only, so they are glitch-free with respect to the inputs.

Verification. The reference model is a behavioural accumulator (running total in cents) rather than a second FSM, so it is independent of the state encoding. Directed: N-N-N, D-N, N-D must each vend with no change; D-D must vend with change (expects 4 dispenses, 1 change); both coins at once; a coin during the dispense cycle. Then 4000 cycles of random coins with random resets, comparing both outputs every cycle.

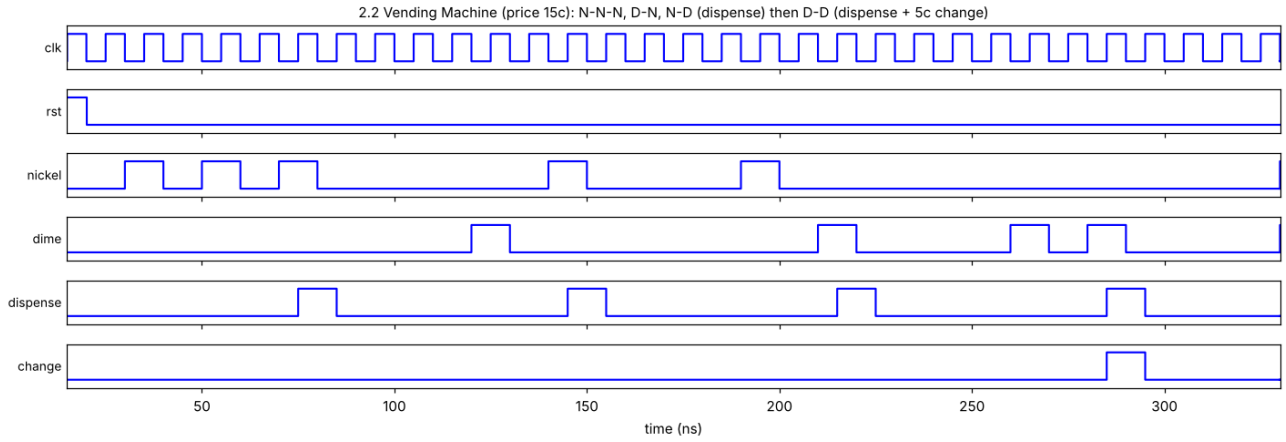
Result. 4039 cycles, 726 dispenses, 267 change returns, 0 errors. Mutation check: removing the change state caused 268 errors; making N+N stall at 5c caused 366 errors.

Issues / notes. Dropping coins during the dispense cycle is a spec decision, not a bug, but a real machine would add a `ready` output or buffer the coin. There is no cancel/refund input.

Simulation output

```
directed: dispenses=4 (expect 4) changes=1 (expect 1)
VENDING TEST: 4039 cycles, 726 dispenses, 267 change, 0 errors -> PASS
```

Waveform



2.2 RTL source (vending.v)

```
// 2.2 Vending Machine Controller: item price 15 cents, accepts nickel(5)/dime(10), Moore FSM
module vending (
    input  wire clk, rst,
    input  wire nickel, dime,       // one coin per cycle (both high = invalid, ignored)
    output wire dispense,           // 1-cycle pulse
    output wire change              // 1-cycle pulse: return a nickel (5c)
);
    localparam C0=3'd0, C5=3'd1, C10=3'd2, VEND=3'd3, VEND_CHG=3'd4;
    reg [2:0] state, next;
    wire coin_n = nickel & ~dime, coin_d = dime & ~nickel;
    always @(posedge clk) state <= rst ? C0 : next;
    always @* begin
        next = state;
        case (state)
            C0:  if (coin_n) next = C5;   else if (coin_d) next = C10;
            C5:  if (coin_n) next = C10;  else if (coin_d) next = VEND;       // 15
            C10: if (coin_n) next = VEND; else if (coin_d) next = VEND_CHG;   // 20
            VEND, VEND_CHG: next = C0;     // coins ignored while dispensing
            default: next = C0;
        endcase
    end
    assign dispense = (state == VEND) | (state == VEND_CHG);   // Moore outputs
    assign change   = (state == VEND_CHG);
endmodule
```

2.2 Testbench (vending_tb.v)

```
`timescale 1ns/1ps
module vending_tb;
    reg clk=0, rst=1, nickel=0, dime=0; wire dispense, change;
    vending dut(clk,rst,nickel,dime,dispense,change);
    always #5 clk=~clk;
    // behavioural reference model (running total in cents), independent of the FSM encoding
    integer tot = 0; reg pend_d = 0, pend_c = 0, check_en = 0;
    integer errors=0, cycles=0, n_disp=0, n_chg=0, i, tn;
    wire coin_valid = nickel ^ dime; integer coin_val;
    always @(posedge clk) begin
        coin_val = nickel ? 5 : 10;
        if (rst) begin tot=0; pend_d=0; pend_c=0; check_en=1; end
        else if (check_en) begin
            cycles = cycles + 1;
            if (dispense !== pend_d || change !== pend_c) begin
                errors=errors+1; $display("FAIL t=%0t disp=%b(exp %b) chg=%b(exp %b)",$time,dispense,pend_d,change,pend_c);
            end
            if (dispense) n_disp = n_disp + 1;  if (change) n_chg = n_chg + 1;
            if (pend_d) begin tot=0; pend_d=0; pend_c=0; end         // dispensing cycle: coins ignored
            else if (coin_valid) begin
                tn = tot + coin_val;
                if (tn >= 15) begin pend_d = 1; pend_c = (tn == 20); end
                tot = tn;
            end
        end
    end
    task coin(input is_dime); begin @(negedge clk); nickel = ~is_dime; dime = is_dime; @(negedge clk); nickel=0; dime=0; end end
    task idle(input integer n); begin repeat(n) @(negedge clk); end endtask
    initial begin
        $dumpfile("vending.vcd"); $dumpvars(0, vending_tb);
        repeat(2) @(negedge clk); rst=0;
        coin(0); coin(0); coin(0); idle(3);   // N N N  -> vend, no change
        coin(1); coin(0);          idle(3);   // D N    -> vend, no change
        coin(0); coin(1);          idle(3);   // N D    -> vend, no change
        coin(1); coin(1);          idle(3);   // D D    -> vend + change
        $display("directed: dispenses=%0d (expect 4) changes=%0d (expect 1)", n_disp, n_chg);
        if (n_disp != 4 || n_chg != 1) errors = errors + 1;
        // both coin inputs high is invalid -> ignored
        @(negedge clk); nickel=1; dime=1; @(negedge clk); @(negedge clk); nickel=0; dime=0; idle(2);
        // coin arriving in the dispensing cycle is ignored (by design)
        coin(1); @(negedge clk); nickel=0; dime=1; @(negedge clk); nickel=1; dime=0; @(negedge clk); nickel=0; idle(4);
        // random traffic with occasional reset
        for (i=0;i<4000;i=i+1) begin
            @(negedge clk); nickel=$random; dime=$random; rst = ($urandom%400==0);
        end
        @(negedge clk); rst=0; nickel=0; dime=0; idle(3);
        $display("VENDING TEST: %0d cycles, %0d dispenses, %0d change, %0d errors -> %s",cycles,n_disp,n_chg,errors,errors==0?"P
        $finish;
    end
endmodule
```

2.3 Synchronous FIFO

Design approach. Parameterized (`DATA_W`, `DEPTH`, power of two). Read and write pointers are one bit wider than the address; **empty** = pointers equal, **full** = pointers equal except the wrap bit, and **count** = wptr - rptr. A write is accepted only if not full and a read only if not empty; a rejected write/read raises a one-cycle `overflow`/`underflow` pulse. Simultaneous read and write is supported. Read data is registered, so `rdata` is valid in the cycle after an accepted read, flagged by `rvalid`.

Verification. A scoreboard model (circular array plus occupancy counter) predicts full, empty, count, overflow, underflow, rvalid and rdata every cycle. Directed phases: read from empty, fill to full and keep writing, drain and keep reading, simultaneous read/write. Then random traffic in write-heavy, balanced and read-heavy phases (80/20, 50/50, 20/80 write probability) with a mid-run reset. The same testbench was run with three configurations by overriding parameters on the command line.

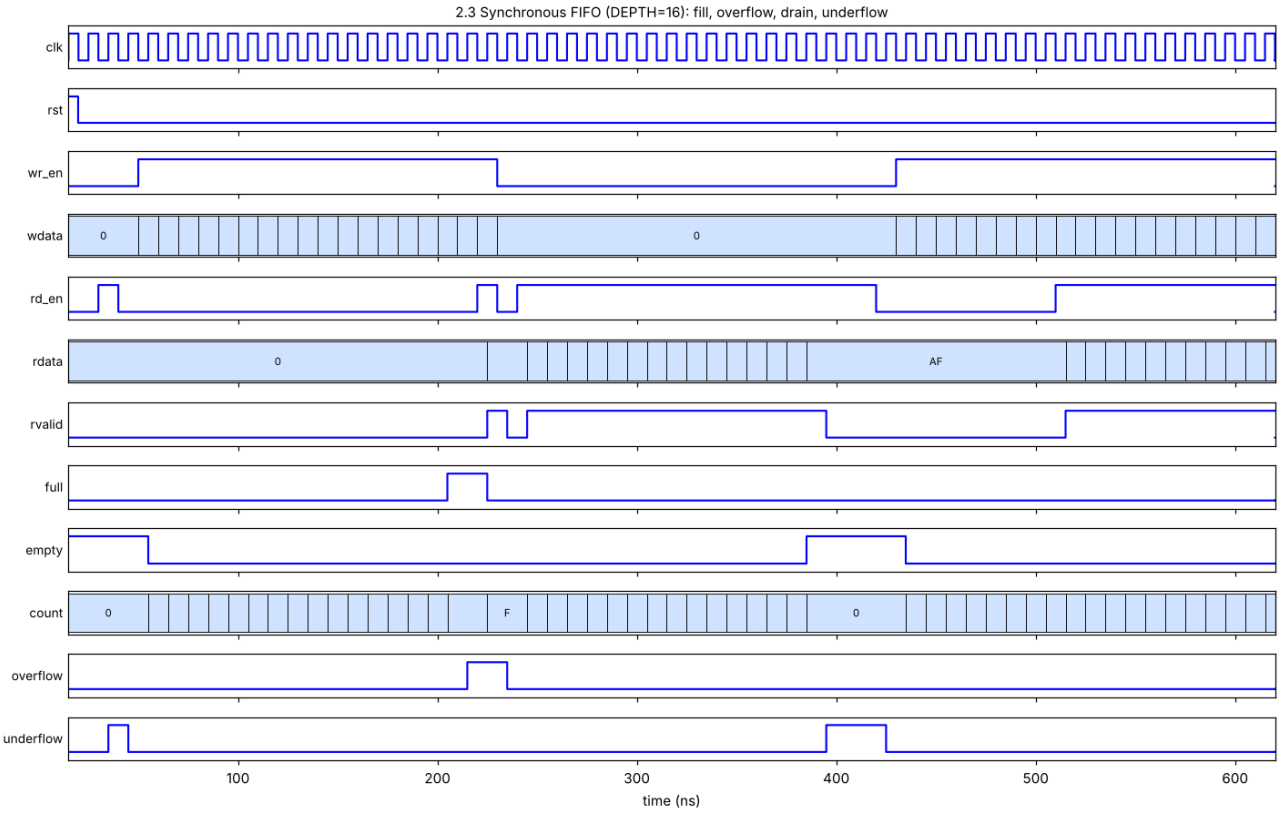
Result. All three configurations pass (see output). Mutation check: allowing writes when full gave 2614 errors; a wrong full flag 3243; a read pointer that skips 4227; a missing underflow flag 992.

Issues / notes. Two things the testbench taught me about the spec: at full, a simultaneous read and write rejects the write (the freed slot is not reused in the same cycle); and read latency is one cycle, so any consumer must use `rvalid`. Overflow/underflow are pulses and will be missed if not sampled.

Simulation output

```
SYNC_FIFO TEST (DATA_W=8 DEPTH=16): 5084 cycles, 1646 writes, 1644 reads, 927 overflow pulses, 992 underflow pulses, 0 errors ->
SYNC_FIFO TEST (DATA_W=8 DEPTH=4): 5030 cycles, 1517 writes, 1515 reads, 1014 overflow pulses, 1085 underflow pulses, 0 errors -
SYNC_FIFO TEST (DATA_W=16 DEPTH=32): 5156 cycles, 1727 writes, 1725 reads, 902 overflow pulses, 959 underflow pulses, 0 errors -
```

Waveform



2.3 RTL source (sync_fifo.v)

```
// 2.3 Synchronous FIFO: parameterized width/depth (depth must be a power of two, >= 2)
// Pointers carry one extra wrap bit -> full/empty without a separate counter.
// Read data is registered: rdata is valid in the cycle after an accepted read (rvalid=1).
module sync_fifo #(parameter DATA_W = 8, parameter DEPTH = 16) (
    input  wire                 clk, rst,
    input  wire                 wr_en,
    input  wire [DATA_W-1:0]    wdata,
    input  wire                 rd_en,
    output reg  [DATA_W-1:0]    rdata,
    output reg                  rvalid,
    output wire                 full, empty,
    output wire [$clog2(DEPTH):0] count,
    output reg                  overflow, underflow   // 1-cycle pulses on rejected write / read
);
    localparam AW = $clog2(DEPTH);
    reg [DATA_W-1:0] mem [0:DEPTH-1];
    reg [AW:0] wptr, rptr;
    assign count = wptr - rptr;
    assign empty = (wptr == rptr);
    assign full  = (wptr[AW] != rptr[AW]) && (wptr[AW-1:0] == rptr[AW-1:0]);
    wire do_wr = wr_en && !full;     // write rejected when full (even if a read happens this cycle)
    wire do_rd = rd_en && !empty;    // read rejected when empty
    always @(posedge clk) begin
        if (rst) begin
            wptr <= 0; rptr <= 0; rvalid <= 0; rdata <= 0; overflow <= 0; underflow <= 0;
        end else begin
            overflow  <= wr_en && full;
            underflow <= rd_en && empty;
            rvalid    <= do_rd;
            if (do_wr) begin mem[wptr[AW-1:0]] <= wdata; wptr <= wptr + 1'b1; end
            if (do_rd) begin rdata <= mem[rptr[AW-1:0]]; rptr <= rptr + 1'b1; end
        end
    end
endmodule
```

2.3 Testbench (sync_fifo_tb.v)

```
`timescale 1ns/1ps
module sync_fifo_tb;
    parameter DW = 8, DEPTH = 16;
    reg clk=0, rst=1, wr_en=0, rd_en=0; reg [DW-1:0] wdata=0;
    wire [DW-1:0] rdata; wire rvalid, full, empty, overflow, underflow; wire [$clog2(DEPTH):0] count;
    sync_fifo #(DW,DEPTH) dut(clk,rst,wr_en,wdata,rd_en,rdata,rvalid,full,empty,count,overflow,underflow);
    always @(posedge clk) ; always #5 clk=~clk;
    // scoreboard / reference model
    reg [DW-1:0] mmem [0:DEPTH-1]; integer mw=0, mr=0, mcount=0;
    reg exp_rvalid=0, exp_ovf=0, exp_unf=0; reg [DW-1:0] exp_rdata=0; reg check_en=0;
    integer errors=0, cycles=0, writes=0, reads=0, ovfs=0, unfs=0, i;
    reg mdo_wr, mdo_rd;
    always @(posedge clk) begin
        if (rst) begin mw=0; mr=0; mcount=0; exp_rvalid=0; exp_ovf=0; exp_unf=0; check_en=1; end
        else if (check_en) begin
            cycles = cycles + 1;
            if (full !== (mcount==DEPTH) || empty !== (mcount==0) || count !== mcount ||
                overflow !== exp_ovf || underflow !== exp_unf || rvalid !== exp_rvalid ||
                (exp_rvalid && rdata !== exp_rdata)) begin
                errors = errors + 1;
                $display("FAIL t=%0t count=%0d(exp %0d) full=%b empty=%b ovf=%b/%b unf=%b/%b rvalid=%b/%b rdata=%h(exp %h)",
                    $time,count,mcount,full,empty,overflow,exp_ovf,underflow,exp_unf,rvalid,exp_rvalid,rdata,exp_rdata);
            end
            if (overflow) ovfs = ovfs + 1; if (underflow) unfs = unfs + 1;
            mdo_wr = wr_en && mcount < DEPTH;  mdo_rd = rd_en && mcount > 0;
            exp_ovf = wr_en && mcount == DEPTH;  exp_unf = rd_en && mcount == 0;
            exp_rvalid = mdo_rd;
            if (mdo_rd) begin exp_rdata = mmem[mr]; mr = (mr+1) % DEPTH; mcount = mcount - 1; reads = reads + 1; end
            if (mdo_wr) begin mmem[mw] = wdata; mw = (mw+1) % DEPTH; mcount = mcount + 1; writes = writes + 1; end
        end
    end
    task cyc(input w, input r, input [DW-1:0] d); begin @(negedge clk); wr_en=w; rd_en=r; wdata=d; end endtask
    integer pw;
    initial begin
        $dumpfile("fifo.vcd"); $dumpvars(0, sync_fifo_tb);
        repeat(2) @(negedge clk); rst=0;
        // 1) underflow on empty
        cyc(0,1,0); cyc(0,0,0);
        // 2) fill completely, then overflow attempts
        for (i=0;i<DEPTH;i=i+1) cyc(1,0,i+8'hA0);
        cyc(1,0,8'hEE); cyc(1,1,8'hEF);       // write rejected at full (read accepted)
        cyc(0,0,0);
        // 3) drain completely, then underflow
        for (i=0;i<DEPTH+2;i=i+1) cyc(0,1,0);
        cyc(0,0,0);
        // 4) simultaneous read+write in the middle (data should flow through in order)
        for (i=0;i<DEPTH/2;i=i+1) cyc(1,0,i+8'h10);
        for (i=0;i<DEPTH*2;i=i+1) cyc(1,1,i+8'h40);
        // 5) random phases: write-heavy, balanced, read-heavy, with a mid-test reset
        for (pw=80; pw>=20; pw=pw-30) begin
            for (i=0;i<1500;i=i+1) cyc(($urandom%100)<pw, ($urandom%100)<(100-pw), $random);
        end
        @(negedge clk); rst=1; @(negedge clk); rst=0;
        for (i=0;i<500;i=i+1) cyc($random, $random, $random);
        cyc(0,0,0); cyc(0,0,0); cyc(0,0,0);
        $display("SYNC_FIFO TEST (DATA_W=%0d DEPTH=%0d): %0d cycles, %0d writes, %0d reads, %0d overflow pulses, %0d underflow p
                 DW,DEPTH,cycles,writes,reads,ovfs,unfs,errors,errors==0?"PASS":"FAIL");
        $finish;
    end
endmodule
```

2.4 Design Review

A self-review of the three Day 2 designs using four kinds of evidence: lint, synthesis, mutation testing of the testbenches, and a manual checklist. The workbook's own review template was not available, so this follows common RTL review practice.

Review checklist

Check	2.1 Seq. detector	2.2 Vending	2.3 FIFO
Reset style consistent and complete	Pass (sync, state)	Pass (sync, state)	Pass (sync, pointers/flags). Memory array not reset (intended)
No inferred latches (Yosys *check*)	Pass	Pass	Pass
FSM has default / all states reachable	Pass	Pass	n/a
Output timing	Mealy, combinational from din (glitch risk if din is late)	Moore, state-decoded (clean)	full/empty combinational from pointers; rdata registered
Blocking vs non-blocking	Pass: <= in clocked, = in combinational	Pass	Pass
Parameterization / reuse	Fixed pattern	Fixed price	Parameters OK; **DEPTH must be a power of two and is not checked**
Illegal input handling	n/a	Both coins = ignored (tested)	Overflow/underflow pulses (tested)
Lint (iverilog -Wall)	Only a 'no timescale' notice	Same	Clean (RTL compiled alone)

Synthesis evidence (Yosys generic *synth*)

Design	Total cells	Flip-flops	Notes
seq_detector	16	4	2-bit declared state; 4 flops are consistent with Yosys re-encoding the FSM one-hot
vending	27	5	5 states, 5 flops (one-hot style)
sync_fifo 8x16	360	149 (128 data + 21 control)	120 muxes for the read port. Generic synthesis maps the memory to flip-flops; a FPGA flow would normally infer RAM (not run here)

These are technology-independent cell counts, not LUT or timing numbers; a Vivado run would be needed for those.

Testbench quality: mutation analysis

To check that each testbench can actually fail, bugs were injected into a copy of each design and the testbench re-run. All eight were caught, which gives more confidence than the PASS lines alone.

Design	Injected bug	Errors reported
2.1	S3 on '0' goes to S0 instead of S2 (overlap wrong)	39
2.1	Output ignores din	177
2.2	No change returned on dime+dime	268
2.2	Nickel in C5 does not advance	366
2.3	Write accepted when full	2614
2.3	full flag ignores address bits	3243
2.3	Read pointer increments by 2	4227
2.3	Underflow flag never raised	992

Findings and recommended improvements

#	Finding	Severity	Recommendation
1	Seq. detector output is a combinational function of the input	Medium if the output feeds other clocked logic or the input is asynchronous	Register the output (1 cycle latency) or synchronize the input
2	Vending machine silently drops a coin that arrives in the dispense cycle; price and coin values are hard-coded; no refund	Low (documented spec choice)	Add a ready/busy output, make PRICE a parameter, add a cancel input

#	Finding	Severity	Recommendation
3	FIFO assumes DEPTH is a power of two but does not check it	Medium (silent failure for other values)	Add an elaboration-time check, e.g. an initial block that errors if DEPTH & (DEPTH-1) != 0
4	FIFO overflow/underflow are one-cycle pulses	Low	Make them sticky with a clear, or count events
5	No almost-full / almost-empty flags	Low	Add programmable thresholds if used for flow control
6	Reset style differs from Day 1 (async active-low) vs Day 2 (sync active-high)	Low (style)	Pick one convention per project
7	No coverage metrics collected (no coverage tool in this environment)	Process gap	Report state/transition coverage in a full simulator, or argue it from the directed tests

Day 3: Extending the Single-Cycle RISC-V CPU

Architecture. A classic single-cycle RV32I processor: each instruction is fetched, decoded, executed, and written back within one clock cycle (CPI = 1). The design is split into six modules in `rv_cpu.v`: `rv_alu`, `rv_regfile` (32 x 32, two read ports, x0 hard-wired to zero), `rv_imem` and `rv_dmem` (256 words each), and `rv_cpu` (immediate generator, control unit, branch comparator, next-PC mux, write-back mux). Data memory supports word loads/stores only.

Design note: register file without bypass. The Day 1 register file has a same-cycle write-through bypass. It cannot be reused here: for an instruction such as `addi x1,x1,1` the write data depends on the read data, so a bypass creates a combinational loop. The CPU therefore uses a version without bypass (reads see the old value, the write lands on the clock edge).

3.1 Instruction Extensions

The base CPU implements a minimal teaching subset; section 3.1 extends it. Because the workbook's list was not available, the extension set below covers every remaining RV32I integer instruction except byte/halfword memory access, fences and system calls. Branch/jump additions that belong to 3.5 (additional) were needed by the assembly programs, so they are included and marked.

Group	Instructions	Encoding	What changed in the RTL
Base (given)	add sub and or slt | lw sw | beq | addi | jal	R: 0110011, I: 0010011/0000011, S: 0100011, B: 1100011, J: 1101111	Original datapath
R-type logic / shift / compare	**xor sll srl sra sltu**	0110011; funct3 100/001/101/101/011, funct7[5] selects SRA	3 new ALU ops (XOR, SLL/SRL/SRA with shamt = b[4:0], SLTU); funct3 decoder
I-type ALU	**andi ori xori slti sltiu slli srli srai**	0010011; same funct3 as R-type; funct7[5] selects SRAI	Reuse of the funct3 decoder; ALU B input muxed to the I-immediate; funct7[5] honoured only where it is part of the opcode (SRAI, SUB) so a negative immediate cannot turn ADDI into SUB
Upper immediates	**lui auipc**	0110111 / 0010111, U-type immediate	New U-type immediate; ALU A input mux: rs1 / pc / zero (a_sel). LUI = 0 + imm, AUIPC = pc + imm
Branches (3.5)	**bne blt bge bltu bgeu**	1100011; funct3 001/100/101/110/111	Branch comparator produces eq, lt (signed), ltu; funct3 selects condition and inversion
Jumps (3.5)	**jalr**	1100111, I-type	Target = (rs1 + imm) & ~1 from the ALU; write-back mux selects pc+4 (also for jal)

Control signals by instruction class

Class	reg_we	b_imm	a_sel	wb_sel	mem_we	next pc
R-type	1	0 (rs2)	rs1	alu	0	pc+4
I-type ALU	1	1 (imm_i)	rs1	alu	0	pc+4
lw	1	1 (imm_i)	rs1	mem	0	pc+4
sw	0	1 (imm_s)	rs1	-	1	pc+4
branch	0	0	rs1	-	0	pc+imm_b if taken else pc+4
jal	1	-	-	pc+4	0	pc+imm_j
jalr	1	1 (imm_i)	rs1	pc+4	0	(rs1+imm)&~1
lui	1	1 (imm_u)	zero	alu	0	pc+4
auipc	1	1 (imm_u)	pc	alu	0	pc+4

Issues faced / design decisions. (1) Register file bypass loop (above). (2) funct7 bit 5 must be ignored for ADDI/SLTI/etc.: it is the sign bit of the immediate, so using it unconditionally would mis-decode ADDI with a negative immediate as SUB. (3) JALR must clear bit 0 of the target; this turned out to be easy to miss in testing (see 3.3). (4) Unknown opcodes are treated as NOPs rather than trapping.

3.2 Assembly Programs

No RISC-V toolchain was available, so a small two-pass Python assembler (`tools/rvasm.py`) supports the instruction set plus the pseudo-instructions `li, mv, nop, j, halt` (halt = `jal x0,0`, a branch to itself, which the testbench detects). Each program's expected result was derived by hand and checked against an independent Python instruction-set simulator before being used to test the RTL.

Program	Purpose	Static instr.	Executed	Verified result
prog_sum	Loop, add, addi, bge, sw	9	36	Sum 1..10 = 55 (x10, mem[0])
prog_fib	Loop with store/load addressing, blt	14	84	F0..F10 in mem[0..10]; x10 = F10 = 55
prog_fact	Nested loops, multiply by repeated addition, bne, beq	14	81	5! = 120 (x10, mem[0])
prog_minmax	Array in memory, lw, signed compare, negative data	26	46	max 42, min -5
prog_alu_all	Every implemented instruction, all 6 branches taken and not taken, jal, jalr (incl. odd target)	96	88	22 stored ALU results, branch tally 127, call/return 11, odd-target jalr 23

Why prog_alu_all matters. It is the instruction coverage test: it uses 11 R-type, 9 I-type ALU, lui, auipc, lw, sw, 6 branch types in both directions, jal and jalr. Its expected values were derived by hand (for example 100 + (-7) = 93; -7 >> 5 logical = 0x07FFFFFF; arithmetic = -1; auipc at address 0xBC = 0x10BC), not copied from the RTL.

Source: prog_sum.s

```
# Sum of 1..10  -> x10 = 55, mem[0] = 55
        li   x1, 0          # sum
        li   x2, 1          # i
        li   x3, 10         # limit
loop:   add  x1, x1, x2     # sum += i
        addi x2, x2, 1      # i++
        bge  x3, x2, loop   # while (limit >= i)
        sw   x1, 0(x0)
        mv   x10, x1
end:    halt
```

Source: prog_fib.s

```
# Fibonacci F0..F10 stored to mem[0..10]; x10 = F10 = 55
        li   x1, 0          # F(n-2)
        li   x2, 1          # F(n-1)
        li   x5, 0          # byte offset
        li   x6, 11         # count
        li   x7, 0          # i
loop:   sw   x1, 0(x5)      # mem[i] = F(i)
        add  x3, x1, x2     # next
        mv   x1, x2
        mv   x2, x3
        addi x5, x5, 4
        addi x7, x7, 1
        blt  x7, x6, loop
        lw   x10, 40(x0)    # F10
end:    halt
```

Source: prog_fact.s

```
# Factorial of 5 using multiplication by repeated addition (no MUL instruction)
# x10 = 120, mem[0] = 120
        li   x1, 5          # n
        li   x2, 1          # result
outer:  beq  x1, x0, done
        li   x3, 0          # product accumulator
        mv   x4, x1         # counter = n
inner:  add  x3, x3, x2     # product += result  (result * n)
        addi x4, x4, -1
        bne  x4, x0, inner
        mv   x2, x3
        addi x1, x1, -1
        j    outer
done:   sw   x2, 0(x0)
        mv   x10, x2
end:    halt
```

Source: prog_minmax.s

```
# Max and min of array {3, 17, -5, 42, 8}. Array at mem[16..20]; max -> mem[0] and x10, min -> mem[1] and x11
        li   x1, 3
        sw   x1, 64(x0)
        li   x1, 17
        sw   x1, 68(x0)
        li   x1, -5
        sw   x1, 72(x0)
        li   x1, 42
        sw   x1, 76(x0)
        li   x1, 8
        sw   x1, 80(x0)
        li   x5, 64         # pointer
        li   x6, 84         # end pointer
        lw   x10, 0(x5)     # max = a[0]
        mv   x11, x10       # min = a[0]
        addi x5, x5, 4
scan:   bge  x5, x6, fin
        lw   x7, 0(x5)
        bge  x10, x7, nomax
        mv   x10, x7
nomax:  bge  x7, x11, nomin
        mv   x11, x7
nomin:  addi x5, x5, 4
        j    scan
fin:    sw   x10, 0(x0)
        sw   x11, 4(x0)
end:    halt
```

Source: prog_alu_all.s

```
# Exercises every implemented instruction; results stored to mem[0..] (expected values from the ISS)
        li   x1, 100
        li   x2, -7
        li   x3, 0x0F0F
        li   x4, 5
        add  x5, x1, x2
        sw   x5, 0(x0)
        sub  x6, x1, x2
        sw   x6, 4(x0)
        and  x7, x1, x3
        sw   x7, 8(x0)
        or   x8, x1, x3
        sw   x8, 12(x0)
        xor  x9, x1, x3
        sw   x9, 16(x0)
        sll  x10, x1, x4
        sw   x10, 20(x0)
        srl  x11, x2, x4
        sw   x11, 24(x0)
        sra  x12, x2, x4
        sw   x12, 28(x0)
        slt  x13, x2, x1
        sw   x13, 32(x0)
        sltu x14, x2, x1
        sw   x14, 36(x0)
        slt  x15, x1, x2
        sw   x15, 40(x0)
        addi x16, x1, -1
        sw   x16, 44(x0)
        andi x17, x3, 0xFF
        sw   x17, 48(x0)
        ori  x18, x3, 0x70
        sw   x18, 52(x0)
        xori x19, x3, -1
        sw   x19, 56(x0)
        slti x21, x2, 0
        sw   x21, 60(x0)
        sltiu x22, x2, 1
        sw   x22, 64(x0)
        slli x23, x1, 3
        sw   x23, 68(x0)
        srli x24, x2, 28
        sw   x24, 72(x0)
        srai x25, x2, 1
        sw   x25, 76(x0)
        lui  x26, 0xABCDE
        sw   x26, 80(x0)
        auipc x27, 1
        sw   x27, 84(x0)
        lw   x31, 0(x0)          # load back the first stored result
        # ---- branches: taken
        li   x28, 0
        beq  x1, x1, t1
        addi x28, x28, 1000
t1:     addi x28, x28, 1
        bne  x1, x2, t2
        addi x28, x28, 1000
t2:     addi x28, x28, 2
        blt  x2, x1, t3
        addi x28, x28, 1000
t3:     addi x28, x28, 4
        bge  x1, x2, t4
        addi x28, x28, 1000
t4:     addi x28, x28, 8
        bltu x1, x2, t5
        addi x28, x28, 1000
t5:     addi x28, x28, 16
        bgeu x2, x1, t6
        addi x28, x28, 1000
t6:     addi x28, x28, 32
        # ---- branches: not taken
        beq  x1, x2, bad
        bne  x1, x1, bad
        blt  x1, x2, bad
        bge  x2, x1, bad
        bltu x2, x1, bad
        bgeu x1, x2, bad
        addi x28, x28, 64
        j    jtest
bad:    addi x28, x28, 1500
        # ---- jal / jalr
jtest:  jal  x29, sub1           # x29 = return address
        addi x30, x30, 1         # executes after return
        j    fin
sub1:   addi x30, x0, 10
        jalr x0, x29, 0          # return
fin:    sw   x28, 88(x0)
        sw   x29, 92(x0)
        sw   x30, 96(x0)
        # ---- jalr must clear bit 0 of the target: (x6+9)&~1 = x6+8, skipping one instruction
        jal  x6, nxt             # x6 = address of nxt
nxt:    jalr x7, x6, 9           # target has bit0 set; hardware must clear it
```

```
        addi x30, x30, 100       # must be skipped
        addi x30, x30, 5
        addi x30, x30, 7
        auipc x8, 0              # captures pc: exposes an odd (bit0 set) pc if JALR failed to clear it
        sw   x8, 108(x0)
        sw   x7, 100(x0)         # return address (nxt+4)
        sw   x30, 104(x0)
end:    halt
```

Machine code listing for prog_sum (assembler output)

```
0000: 00000093  li   x1, 0
0004: 00100113  li   x2, 1
0008: 00a00193  li   x3, 10
000c: 002080b3  add  x1, x1, x2
0010: 00110113  addi x2, x2, 1
0014: fe21dce3  bge  x3, x2, loop
0018: 00102023  sw   x1, 0(x0)
001c: 00008513  mv   x10, x1
0020: 0000006f  halt
```

3.3 Self-Checking CPU Testbench

Method. For every program the testbench (1) clears registers and data memory, (2) loads the program image with `$readmemh`, (3) resets and runs the CPU until it branches to itself, (4) compares all 31 registers (x1..x31) and the first 64 data-memory words with an expected image produced by the Python ISS, (5) prints per-program PASS/FAIL and a final summary. A timeout stops runaway programs. A per-cycle trace file is written for each program for waveform analysis.

Why this is independent verification. The ISS is a separate implementation written in a different language, and it was itself validated against hand-derived answers (sum = 55, Fibonacci, 5! = 120, min/max, and 22 individual ALU results) before being trusted. The RTL therefore has to agree with a model that has been checked against arithmetic done by hand.

Result

```
prog_sum       halted after 36 cycles (CPI=1 => 36 instructions), 0 mismatches -> PASS
prog_fib       halted after 84 cycles (CPI=1 => 84 instructions), 0 mismatches -> PASS
prog_fact      halted after 81 cycles (CPI=1 => 81 instructions), 0 mismatches -> PASS
prog_minmax    halted after 46 cycles (CPI=1 => 46 instructions), 0 mismatches -> PASS
prog_alu_all   halted after 88 cycles (CPI=1 => 88 instructions), 0 mismatches -> PASS
CPU TESTBENCH: 5 programs, 0 failed, 0 total mismatches -> PASS
```

Mutation analysis of the testbench

Twelve plausible RTL bugs were injected one at a time. All twelve are detected:

Injected bug	Programs failing / total mismatches
BGE condition inverted	3 failed, 12 total mismatches
BLT condition inverted	2 failed, 18 total mismatches
BEQ condition inverted	2 failed, 7 total mismatches
SRA behaves as SRL	1 failed, 4 total mismatches
SLT compares unsigned	1 failed, 6 total mismatches
JALR does not clear target bit 0	1 failed, 2 total mismatches
AUIPC ignores pc	1 failed, 4 total mismatches
SUB decoded as ADD	1 failed, 1 total mismatches
BLTU/BGEU use signed compare	1 failed, 2 total mismatches
JAL uses B-type immediate	2 failed, 14 total mismatches
SRAI/SRA decode lost	1 failed, 4 total mismatches
SW uses I-type immediate	2 failed, 35 total mismatches

Issues faced during verification.

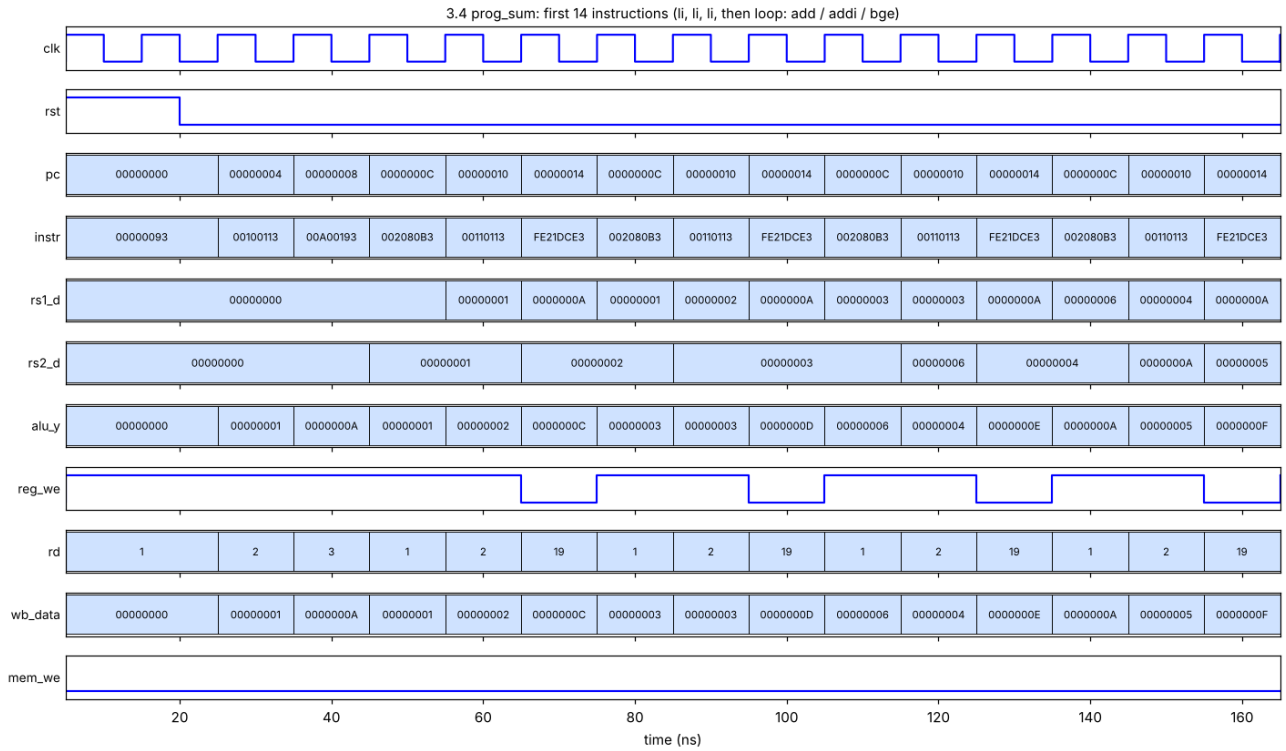
1. A real coverage gap was found and closed. The first version of prog_alu_all did not catch the bug 'JALR does not clear bit 0 of the target'. The reason: with bit 0 left set the pc becomes odd, but the instruction memory ignores the low two address bits, so every fetch still hit the right word and the program ran identically. Only something that *reads the pc* afterwards can expose it. The fix was to add an odd-target `jalr` followed by an `auipc` that captures the pc; the mutant is now caught.

2. Two mutants survive, and both are equivalent (no observable difference), so they are not test gaps: (a) dropping instruction bit 30 from the B-type immediate does not change behaviour because instruction memory is 256 words, so branch offsets never exceed +/-1020 and bit 10 equals the sign bit; (b) allowing writes to x0 is invisible because the register-file read port forces x0 to zero. Both would matter in a larger memory or a different register file; they are noted for the 3.5/Day 4 work.

3. Testbench timing. pc and instr are sampled at the negative clock edge, when all combinational outputs have settled after the previous positive edge, which avoids races between stimulus and checks.

3.4 Waveform Analysis

Program analysed: **prog_sum** (sum of 1 to 10). The waveform and trace table below are taken from the same simulation run that the self-checking testbench verified.



Trace of the first 14 cycles

Cycle	PC	Instr	Assembly	ALU out	we	rd	Write-back	mwe	Next PC
0	0000	00000093	li x1, 0	00000000	1	1	00000000	0	0004
1	0004	00100113	li x2, 1	00000001	1	2	00000001	0	0008
2	0008	00a00193	li x3, 10	0000000a	1	3	0000000a	0	000C
3	000C	002080b3	add x1, x1, x2	00000001	1	1	00000001	0	0010
4	0010	00110113	addi x2, x2, 1	00000002	1	2	00000002	0	0014
5	0014	fe21dce3	bge x3, x2, loop	0000000c	0	(x)	-	0	000C
6	000C	002080b3	add x1, x1, x2	00000003	1	1	00000003	0	0010
7	0010	00110113	addi x2, x2, 1	00000003	1	2	00000003	0	0014
8	0014	fe21dce3	bge x3, x2, loop	0000000d	0	(x)	-	0	000C
9	000C	002080b3	add x1, x1, x2	00000006	1	1	00000006	0	0010
10	0010	00110113	addi x2, x2, 1	00000004	1	2	00000004	0	0014
11	0014	fe21dce3	bge x3, x2, loop	0000000e	0	(x)	-	0	000C
12	000C	002080b3	add x1, x1, x2	0000000a	1	1	0000000a	0	0010
13	0010	00110113	addi x2, x2, 1	00000005	1	2	00000005	0	0014

Column key: we = reg_we, rd = destination register, mwe = mem_we; assembly lines are joined to the simulation log by program counter.

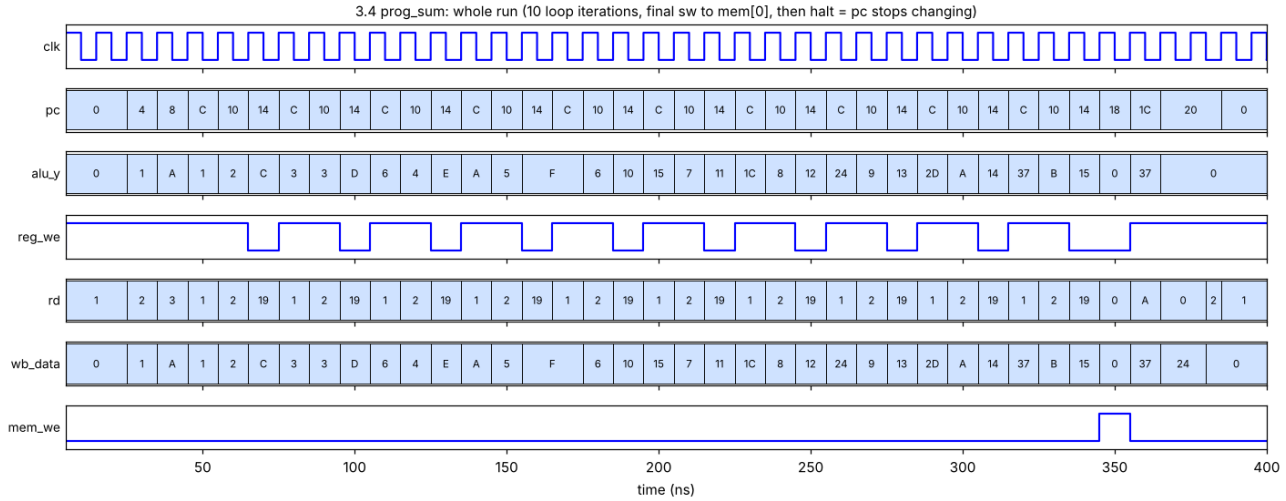
What the waveform shows

1. One instruction per clock. The pc advances by 4 each cycle (0, 4, 8, C, 10, 14) and every instruction completes in that cycle, so CPI = 1: the program executes 36 instructions in 36 cycles. The register write itself happens at the clock edge that ends the cycle, so a value written in cycle N is visible on the read ports from cycle N+1 (for example x1 = 1 is written by the add in cycle 3 and is read as rs1 = 1 by the add in cycle 6).

2. The running sum is visible in the ALU output. At the three add instructions in cycles 3, 6, 9 and 12 the ALU output is 1, 3, 6, 10 (triangular numbers), while the addi in between shows the loop counter incrementing 2, 3, 4, 5.

3. Branch behaviour. The bge at address 0x14 does not write a register (reg_we = 0). The ALU still computes something (x3 + x2, e.g. 0xC = 10 + 2) because the datapath is shared, but the result is ignored. The rd field reads 0x19 (25) only because the branch encoding reuses instruction bits 11:7 for part of the offset; reg_we = 0 is what keeps the register file safe. The branch is taken 9 times, each time sending the pc back from 0x14 to 0x0C.

4. Loop exit and the store. In the last iteration x2 becomes 11, so the condition 10 >= 11 is false and the pc falls through from 0x14 to 0x18, where the sw raises mem_we for one cycle with ALU output 0 (address x0 + 0) and writes 55.



Trace of the last 7 cycles

Cycle	PC	Instr	Assembly	ALU out	we	rd	Write-back	mwe	Next PC
29	0014	fe21dce3	bge x3, x2, loop	00000014	0	(x)	-	0	000C
30	000C	002080b3	add x1, x1, x2	00000037	1	1	00000037	0	0010
31	0010	00110113	addi x2, x2, 1	0000000b	1	2	0000000b	0	0014
32	0014	fe21dce3	bge x3, x2, loop	00000015	0	(x)	-	0	0018
33	0018	00102023	sw x1, 0(x0)	00000000	0	(x)	-	1	001C
34	001C	00008513	mv x10, x1	00000037	1	10	00000037	0	0020
35	0020	0000006f	halt	00000000	1	0	00000024	0	0020

5. Halt. The final instruction is jal x0, 0 at 0x20. Its next pc equals its own address, so the pc stops changing; the testbench detects exactly this condition. Writing to x0 is discarded, so nothing else changes.

6. Timing remark. In an RTL simulation all combinational paths have zero delay, so the waveform cannot show the clock-period limit. In hardware the longest path is a load: instruction memory, register read, ALU address add, data memory read and write-back mux must all fit in one clock. That long critical path is the motivation for pipelining in Day 4.

Appendix: CPU RTL source (rv_cpu.v)

```
// ============================================================================
// Single-cycle RV32I CPU (CPI = 1)  --  Day 3
// Base subset : add sub and or slt lw sw beq addi jal
// 3.1 additions: xor sll srl sra sltu | andi ori xori slti sltiu slli srli srai | lui auipc
//                (also bne blt bge bltu bgeu jalr -- needed by the assembly programs; see 3.5)
// Memory model: word-addressed LW/SW only (no byte/half access, no misalignment trap)
// ============================================================================

// ---- ALU -------------------------------------------------------------------
module rv_alu (
    input  wire [31:0] a, b,
    input  wire [3:0]  op,
    output reg  [31:0] y
);
    localparam ADD=4'd0, SUB=4'd1, AND_=4'd2, OR_=4'd3, XOR_=4'd4,
               SLL=4'd5, SRL=4'd6, SRA=4'd7, SLT=4'd8, SLTU=4'd9;
    always @* begin
        case (op)
            ADD:  y = a + b;
            SUB:  y = a - b;
            AND_: y = a & b;
            OR_:  y = a | b;
            XOR_: y = a ^ b;
            SLL:  y = a << b[4:0];
            SRL:  y = a >> b[4:0];
            SRA:  y = $signed(a) >>> b[4:0];
            SLT:  y = {31'b0, $signed(a) < $signed(b)};
            SLTU: y = {31'b0, a < b};
            default: y = 32'b0;
        endcase
    end
endmodule

// ---- Register file: 2 async read ports, 1 sync write port, x0 = 0, NO bypass ----
// (a bypass would create a combinational loop for e.g. "addi x1,x1,1": wdata depends on rdata)
module rv_regfile (
    input  wire        clk, we,
    input  wire [4:0]  ra1, ra2, wa,
    input  wire [31:0] wd,
    output wire [31:0] rd1, rd2
);
    reg [31:0] mem [0:31];
    integer k; initial for (k=0;k<32;k=k+1) mem[k] = 0;
    always @(posedge clk) if (we && wa != 0) mem[wa] <= wd;
    assign rd1 = (ra1 == 0) ? 32'b0 : mem[ra1];
    assign rd2 = (ra2 == 0) ? 32'b0 : mem[ra2];
endmodule

// ---- Instruction memory (256 words, async read) ----------------------------
module rv_imem (input wire [31:0] addr, output wire [31:0] instr);
    reg [31:0] mem [0:255];
    integer k; initial for (k=0;k<256;k=k+1) mem[k] = 32'h0000_0013;   // NOP (addi x0,x0,0)
    assign instr = mem[addr[9:2]];
endmodule

// ---- Data memory (256 words, async read, sync write) -------------------------
module rv_dmem (input wire clk, we, input wire [31:0] addr, wdata, output wire [31:0] rdata);
    reg [31:0] mem [0:255];
    integer k; initial for (k=0;k<256;k=k+1) mem[k] = 0;
    always @(posedge clk) if (we) mem[addr[9:2]] <= wdata;
    assign rdata = mem[addr[9:2]];
endmodule

// ---- Top: control + datapath -----------------------------------------------
module rv_cpu (input wire clk, rst);
    reg  [31:0] pc;
    wire [31:0] instr;
    rv_imem u_imem (.addr(pc), .instr(instr));

    wire [6:0] opcode = instr[6:0];
    wire [4:0] rd  = instr[11:7], rs1 = instr[19:15], rs2 = instr[24:20];
    wire [2:0] f3  = instr[14:12];
    wire       f7b5 = instr[30];

    // immediate generator
    wire [31:0] imm_i = {{20{instr[31]}}, instr[31:20]};
    wire [31:0] imm_s = {{20{instr[31]}}, instr[31:25], instr[11:7]};
    wire [31:0] imm_b = {{19{instr[31]}}, instr[31], instr[7], instr[30:25], instr[11:8], 1'b0};
    wire [31:0] imm_u = {instr[31:12], 12'b0};
    wire [31:0] imm_j = {{11{instr[31]}}, instr[31], instr[19:12], instr[20], instr[30:21], 1'b0};

    localparam OP_R=7'b0110011, OP_I=7'b0010011, OP_LW=7'b0000011, OP_SW=7'b0100011,
               OP_B=7'b1100011, OP_JAL=7'b1101111, OP_JALR=7'b1100111, OP_LUI=7'b0110111, OP_AUIPC=7'b0010111;

    // ---- control unit ------------------------------------------------------
    reg        reg_we, mem_we, b_imm, is_branch, is_jal, is_jalr;
    reg [1:0]  a_sel;            // 0 rs1, 1 pc, 2 zero
    reg [1:0]  wb_sel;           // 0 alu, 1 mem, 2 pc+4
    reg [31:0] imm;
```

```
    reg [3:0]  alu_op;
    reg [3:0]  funct_op;         // ALU op decoded from funct3/funct7
    always @* begin
        case (f3)
            3'b000: funct_op = (opcode==OP_R && f7b5) ? 4'd1 : 4'd0;   // SUB / ADD
            3'b001: funct_op = 4'd5;                                    // SLL
            3'b010: funct_op = 4'd8;                                    // SLT
            3'b011: funct_op = 4'd9;                                    // SLTU
            3'b100: funct_op = 4'd4;                                    // XOR
            3'b101: funct_op = f7b5 ? 4'd7 : 4'd6;                      // SRA / SRL
            3'b110: funct_op = 4'd3;                                    // OR
            3'b111: funct_op = 4'd2;                                    // AND
        endcase
    end
    always @* begin
        reg_we=0; mem_we=0; b_imm=0; is_branch=0; is_jal=0; is_jalr=0;
        a_sel=0; wb_sel=0; imm=imm_i; alu_op=4'd0;
        case (opcode)
            OP_R:    begin reg_we=1; alu_op=funct_op; end
            OP_I:    begin reg_we=1; b_imm=1; imm=imm_i; alu_op=funct_op; end
            OP_LW:   begin reg_we=1; b_imm=1; imm=imm_i; wb_sel=1; end
            OP_SW:   begin mem_we=1; b_imm=1; imm=imm_s; end
            OP_B:    begin is_branch=1; end
            OP_JAL:  begin reg_we=1; is_jal=1; wb_sel=2; end
            OP_JALR: begin reg_we=1; is_jalr=1; b_imm=1; imm=imm_i; wb_sel=2; end
            OP_LUI:  begin reg_we=1; b_imm=1; imm=imm_u; a_sel=2; end
            OP_AUIPC:begin reg_we=1; b_imm=1; imm=imm_u; a_sel=1; end
            default: ;       // unknown opcode -> behaves as NOP
        endcase
    end

    // ---- datapath -----------------------------------------------------------
    wire [31:0] rs1_d, rs2_d, wb_data, alu_y, dmem_rd;
    wire [31:0] alu_a = (a_sel==0) ? rs1_d : (a_sel==1) ? pc : 32'b0;
    wire [31:0] alu_b = b_imm ? imm : rs2_d;
    rv_alu u_alu (.a(alu_a), .b(alu_b), .op(alu_op), .y(alu_y));
    rv_regfile u_rf (.clk(clk), .we(reg_we), .ra1(rs1), .ra2(rs2), .wa(rd), .wd(wb_data), .rd1(rs1_d), .rd2(rs2_d));
    rv_dmem u_dmem (.clk(clk), .we(mem_we), .addr(alu_y), .wdata(rs2_d), .rdata(dmem_rd));

    wire [31:0] pc4 = pc + 32'd4;
    assign wb_data = (wb_sel==0) ? alu_y : (wb_sel==1) ? dmem_rd : pc4;

    // branch comparator
    wire eq  = (rs1_d == rs2_d);
    wire lt  = ($signed(rs1_d) < $signed(rs2_d));
    wire ltu = (rs1_d < rs2_d);
    reg  take;
    always @* begin
        case (f3)
            3'b000: take = eq;   3'b001: take = ~eq;
            3'b100: take = lt;   3'b101: take = ~lt;
            3'b110: take = ltu;  3'b111: take = ~ltu;
            default: take = 1'b0;
        endcase
    end
    wire [31:0] next_pc = (is_branch && take) ? pc + imm_b :
                          is_jal              ? pc + imm_j :
                          is_jalr             ? (alu_y & 32'hFFFF_FFFE) : pc4;
    always @(posedge clk) pc <= rst ? 32'b0 : next_pc;
endmodule
```

Appendix: CPU testbench (rv_cpu_tb.v)

```
`timescale 1ns/1ps
// Self-checking testbench: for each program -> load imem, reset, run until the CPU
// branches to itself (halt), then compare x1..x31 and dmem[0..63] with the ISS-generated .exp file.
module rv_cpu_tb;
    reg clk = 0, rst = 1;
    rv_cpu dut (.clk(clk), .rst(rst));
    always #5 clk = ~clk;

    // probes (top-level wires so they show up in the VCD)
    wire [31:0] pc      = dut.pc;
    wire [31:0] instr   = dut.instr;
    wire [31:0] alu_y   = dut.alu_y;
    wire        reg_we  = dut.reg_we;
    wire [4:0]  rd      = dut.rd;
    wire [31:0] wb_data = dut.wb_data;
    wire        mem_we  = dut.mem_we;
    wire [31:0] rs1_d   = dut.rs1_d;
    wire [31:0] rs2_d   = dut.rs2_d;

    reg [31:0] expect_mem [0:95];       // 32 registers followed by 64 data words
    integer total_errors = 0, prog_errors, cyc, i, f, nprog = 0, nfail = 0;
    reg [31:0] pc_last; reg halted;
    string hexf, expf, trcf;

    task run_prog(input string name);
        begin
            hexf = {"prog/", name, ".hex"}; expf = {"prog/", name, ".exp"}; trcf = {"prog/", name, ".trace"};
            // clean state, load program + expected image
            for (i = 0; i < 256; i = i + 1) begin dut.u_dmem.mem[i] = 0; end
            for (i = 0; i < 32;  i = i + 1) begin dut.u_rf.mem[i] = 0; end
            $readmemh(hexf, dut.u_imem.mem);
            $readmemh(expf, expect_mem);
            f = $fopen(trcf, "w");
            rst = 1; @(negedge clk); @(negedge clk); rst = 0;
            cyc = 0; halted = 0; pc_last = 32'hFFFF_FFFF;
            while (!halted && cyc < 20000) begin
                // pc/instr are stable here (negedge, after the previous posedge)
                $fdisplay(f, "%0d %h %h %h %b %0d %h %b", cyc, pc, instr, alu_y, reg_we, rd, wb_data, mem_we);
                if (pc === pc_last) halted = 1;            // jal x0,0 : pc did not advance
                pc_last = pc;
                if (!halted) begin @(negedge clk); cyc = cyc + 1; end
            end
            $fclose(f);
            prog_errors = 0;
            if (!halted) begin prog_errors = prog_errors + 1; $display("  %s: TIMEOUT (no halt)", name); end
            for (i = 1; i < 32; i = i + 1)
                if (dut.u_rf.mem[i] !== expect_mem[i]) begin
                    prog_errors = prog_errors + 1;
                    $display("  %s: x%0d = %h, expected %h", name, i, dut.u_rf.mem[i], expect_mem[i]);
                end
            for (i = 0; i < 64; i = i + 1)
                if (dut.u_dmem.mem[i] !== expect_mem[32+i]) begin
                    prog_errors = prog_errors + 1;
                    $display("  %s: mem[%0d] = %h, expected %h", name, i, dut.u_dmem.mem[i], expect_mem[32+i]);
                end
            nprog = nprog + 1; if (prog_errors != 0) nfail = nfail + 1;
            total_errors = total_errors + prog_errors;
            $display("%-14s halted after %0d cycles (CPI=1 => %0d instructions), %0d mismatches -> %s",
                     name, cyc, cyc, prog_errors, prog_errors == 0 ? "PASS" : "FAIL");
        end
    endtask

    initial begin
        $dumpfile("cpu.vcd"); $dumpvars(0, rv_cpu_tb);
        run_prog("prog_sum");
        run_prog("prog_fib");
        run_prog("prog_fact");
        run_prog("prog_minmax");
        run_prog("prog_alu_all");
        $display("CPU TESTBENCH: %0d programs, %0d failed, %0d total mismatches -> %s",
                 nprog, nfail, total_errors, total_errors == 0 ? "PASS" : "FAIL");
        $finish;
    end
endmodule
```